

CM1: Introduction aux Systèmes d'Exploitation



Objectifs du chapitre

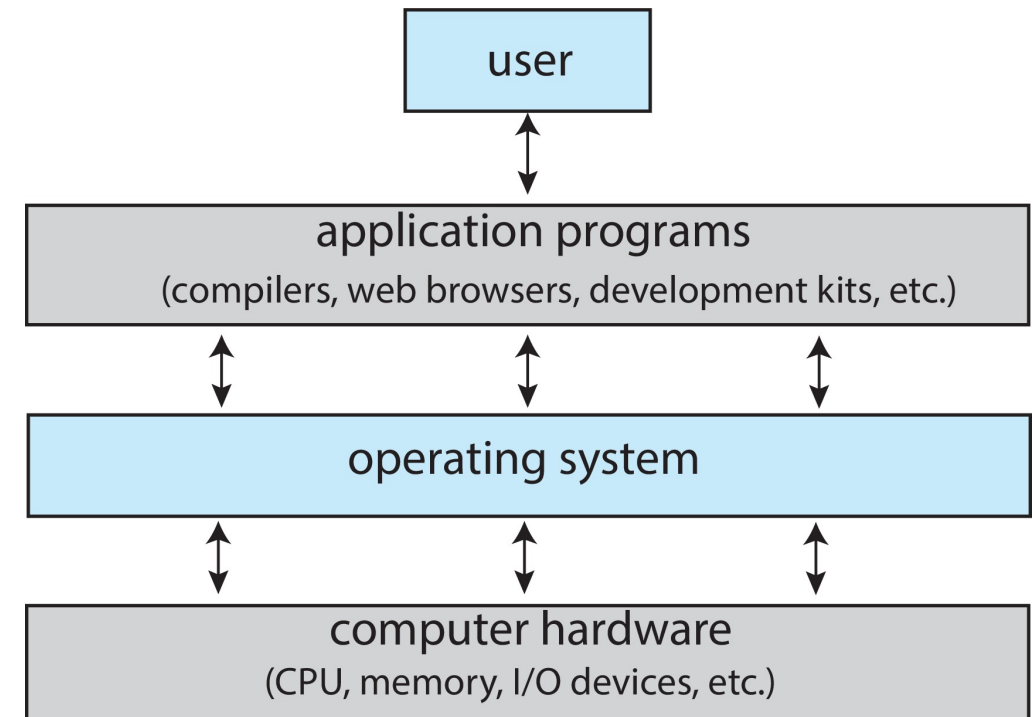
1. Décrire les quatre composants principaux d'un système informatique
2. Expliquer le mécanisme d'interruption et son rôle dans les entrées-sorties
3. Distinguer le mode utilisateur du mode noyau
4. Comparer les architectures monoprocesseur, multiprocesseur, multicœur et cluster
5. Identifier les sous-systèmes de gestion de ressources du SE
6. Retracer l'évolution historique des SE
7. Reconnaître les environnements de calcul modernes (mobile, cloud, embarqué)

Qu'est-ce qu'un système d'exploitation (SE) ?

- **Système d'exploitation (SE)** : logiciel qui gère le matériel informatique, coordonne l'utilisation des ressources entre les applications, et fournit un environnement pour l'exécution des programmes (**Silberschatz**)
- **Rôles principaux** :
 1. **Allocateur de ressources** : gère CPU, mémoire, périphériques
 2. **Programme de contrôle** : supervise l'exécution des programmes utilisateur
- **Analogie** : un SE est comme un gouvernement — il ne produit rien par lui-même mais crée l'environnement où d'autres programmes peuvent travailler

Les quatre couches d'un système informatique

- **Matériel** : CPU, mémoire, dispositifs d'E/S
- **Système d'exploitation** : noyau, services, gestion des ressources
- **Programmes applicatifs** : compilateurs, navigateurs, traitements de texte
- **Utilisateurs** : personnes, autres systèmes



Abstract view of the components of a computer system.

Point de vue utilisateur vs. point de vue système

Point de vue	Objectif principal	Priorités
Utilisateur (PC portable)	Facilité d'utilisation, performance	1. Confort 2. Performance 3. Utilisation des ressources : négligeable
Système (serveur)	Efficacité, équité, protection	1. Taux d'utilisation 2. Équité 3. Sécurité

- **SE embarqué** : peu ou pas d'utilisateur direct, fiabilité avant tout
- **SE mobile** : autonomie énergétique, interface tactile

Définir le SE : noyau, programmes système, middleware

- **Noyau (kernel)** : programme toujours en mémoire, s'exécute en mode privilégié, gère le matériel
- **Programmes système** : utilitaires associés au SE sans être dans le noyau (compilateurs, éditeurs, interpréteurs de commandes)
- **Middleware** : couche de services supplémentaires pour les développeurs (bases de données, multimedia, graphismes)

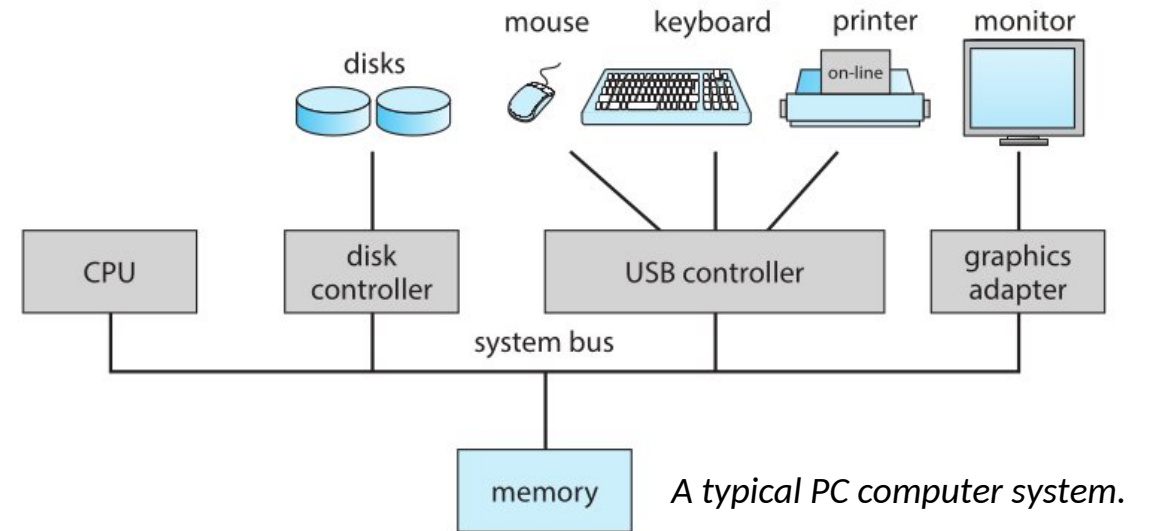
Le SE inclut le noyau + programmes système + middleware

- **Exemple** : Android contient le noyau Linux + bibliothèques + machine virtuelle Dalvik/ART

Section 1.2 — Organisation d'un système informatique

Architecture matérielle d'un ordinateur moderne

- **CPU** : unité centrale de traitement
- **Bus système** : voie de communication principale
- **Mémoire (RAM)** : stockage temporaire et rapide
- **Contrôleurs de périphériques** : disque, USB, graphique



1. Chaque contrôleur gère un type spécifique de périphérique
2. Le SE utilise des pilotes (**device drivers**) pour chaque contrôleur

le SE doit coordonner tous ces éléments.
Un mécanisme central pour cette coordination est l'interruption.

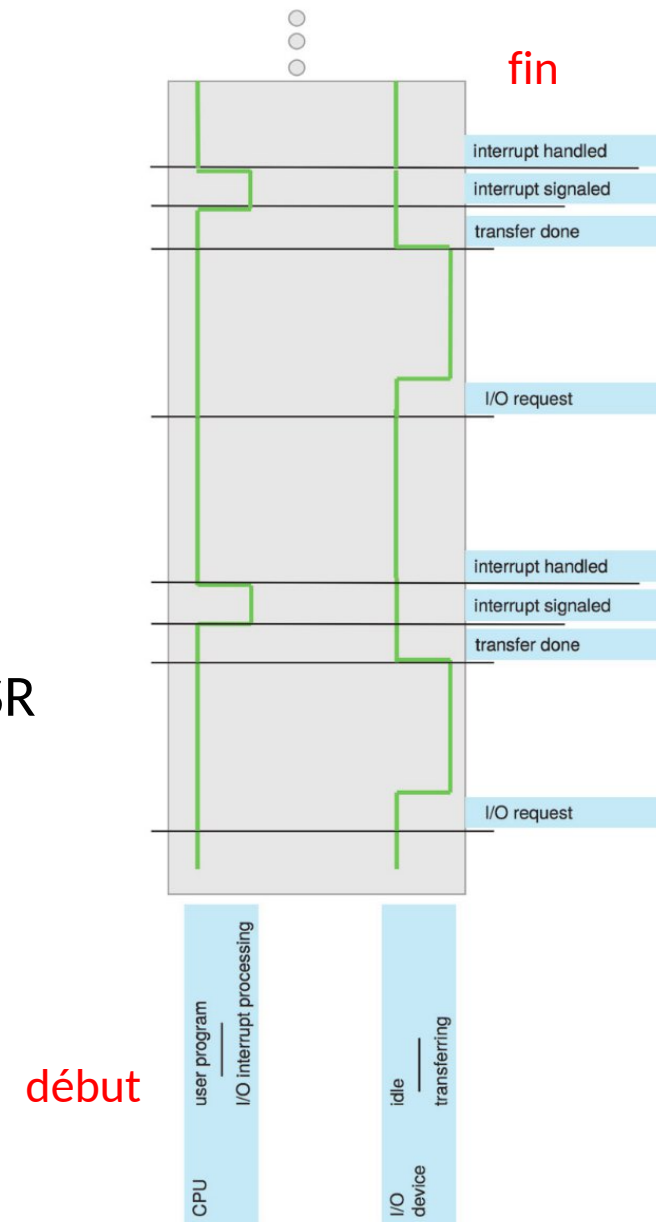
Le mécanisme d'interruption

- **Interruption** : signal envoyé au CPU par un périphérique pour indiquer un événement
- Séquence typique :
 1. Le périphérique termine une opération
 2. Il envoie un signal d'interruption via le bus système
 3. Le CPU suspend l'exécution en cours
 4. Le CPU sauvegarde l'état et exécute le gestionnaire d'interruption (ISR)
 5. Le CPU restaure l'état et reprend l'exécution
- L'interruption évite le *polling* (attente active coûteuse)

l'interruption n'est pas une exception ou une erreur — c'est le moyen normal de gérer les entrées-sorties asynchrones.

Chronologie d'une interruption

1. Le programme utilisateur s'exécute
2. Le périphérique termine une opération d'E/S
3. Le périphérique émet une interruption
4. Le CPU sauvegarde le contexte et charge l'adresse de l'ISR
5. Le gestionnaire d'interruption traite l'événement
6. Le CPU restaure le contexte
7. Le programme utilisateur reprend



Interrupt timeline for a single program doing output

Vecteur d'interruption et gestion

- **Vecteur d'interruption** : tableau en mémoire basse contenant les adresses des gestionnaires
- Chaque type d'interruption possède un numéro unique (0-255
ex: sur Intel)
- Le CPU utilise le numéro comme index dans le vecteur
- **Priorités** : certaines interruptions peuvent en masquer d'autres
- Interruptions masquables (défaut matériel) vs. non masquables (défaut mémoire)
- **Interrupt chaining** : chaînage de gestionnaires pour gérer plus de périphériques que d'entrées dans le vecteur

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19-31	(Intel reserved, do not use)
32-255	maskable interrupts

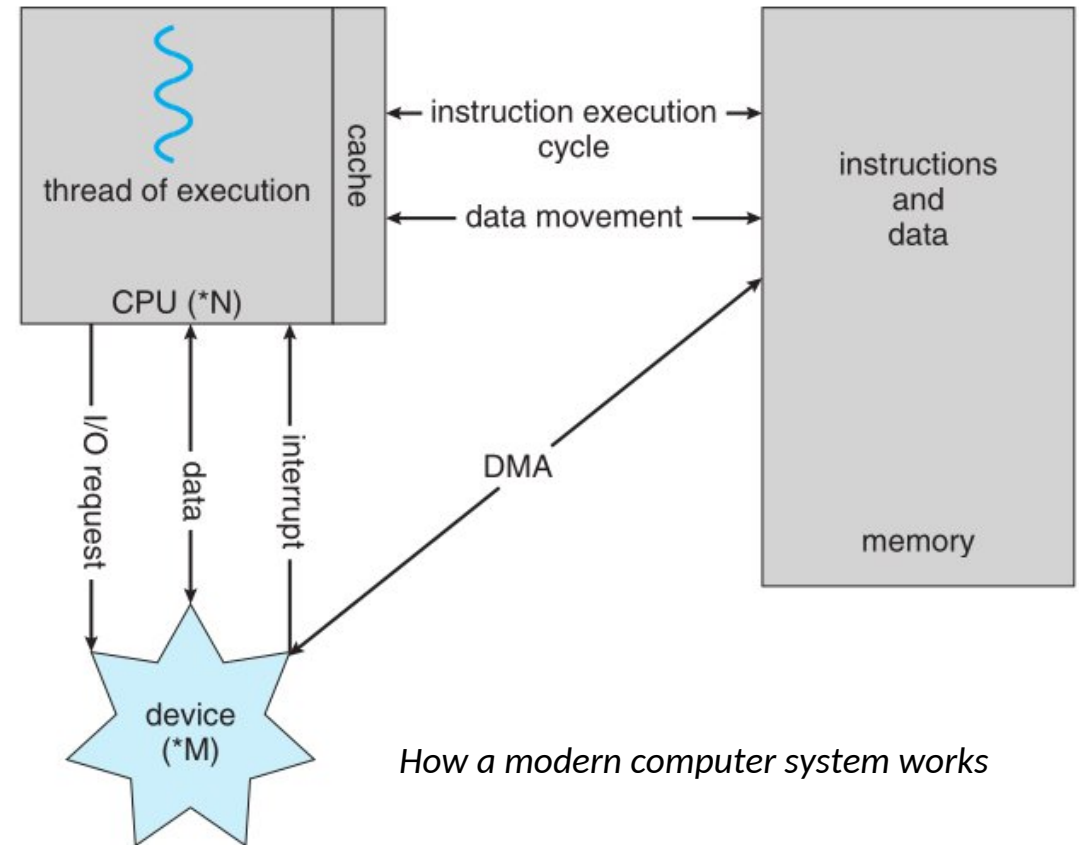
Intel processor event-vector table

Accès Direct Mémoire (DMA)

- **DMA** : mécanisme matériel qui permet à un périphérique de transférer des données directement en mémoire sans intervention du CPU
- **Problème** : les interruptions par octet sont trop coûteuses pour les gros transferts
- Solution avec DMA :
 1. Le CPU configure le transfert (tampon, taille, direction)
 2. Le contrôleur DMA transfère les données
 3. Une seule interruption est émise à la fin du bloc
- Le CPU peut exécuter d'autres tâches pendant le transfert

Synthèse : fonctionnement d'un système moderne

1. **Cycle d'exécution des instructions :**
chargement → décodage → exécution → stockage
2. **Requête d'E/S :** le programme demande une lecture disque
3. **Données :** transférées par DMA du disque vers la mémoire
4. **Interruption :** signalée à la fin du transfert
5. **Traitement :** le pilote de périphérique notifie le programme



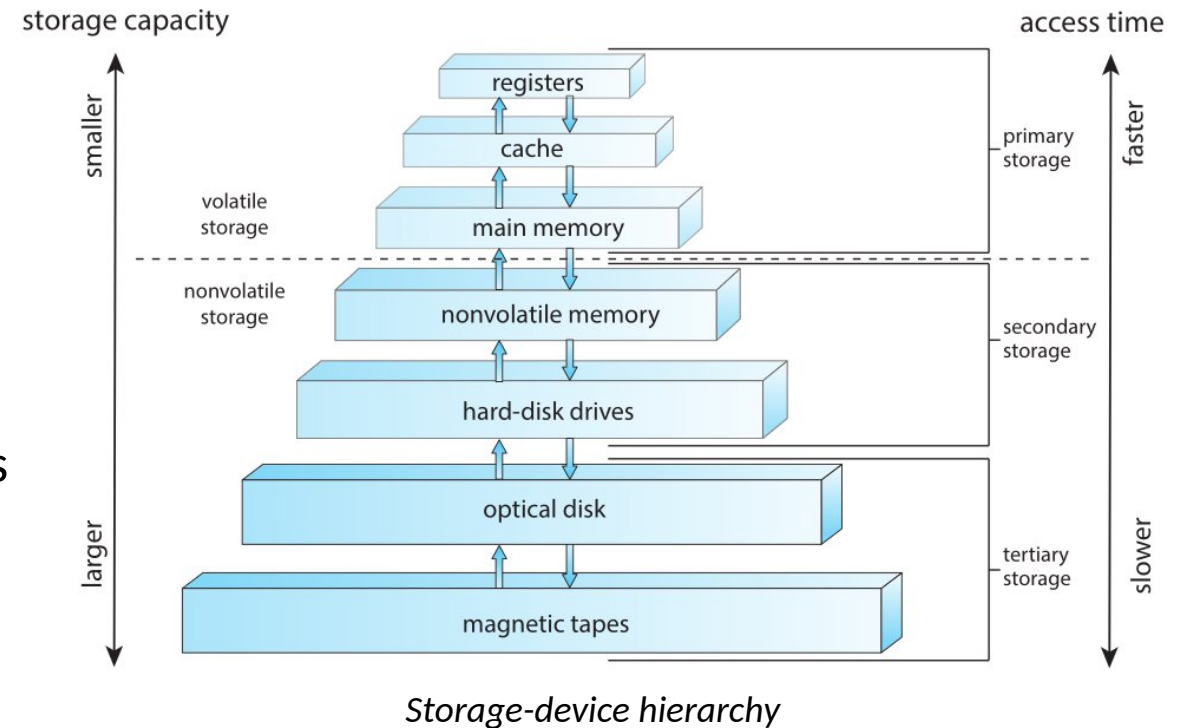
How a modern computer system works

Section 1.3 — Structure de stockage et E/S

La pyramide de la mémoire

- **Registres** (quelques Ko) — vitesse : $\sim 0,25\text{--}0,5$ ns
- **Cache L1/L2** (< 16 Mo) — vitesse : $\sim 0,5\text{--}25$ ns
- **Mémoire vive** (RAM) (< 64 Go) — vitesse : $\sim 80\text{--}250$ ns
- **SSD / NVM** (< 1 To) — vitesse : $\sim 25\text{--}50$ μs
- **Disque dur** (HDD) (< 10 To) — vitesse : $\sim 5\text{--}10$ ms

En montant : plus rapide, plus cher, plus petit



Le rôle du SE est de gérer ces données pour que les accès les plus fréquents restent dans les niveaux rapides — c'est le principe de localité.

Volatilité et persistance

- **Mémoire volatile** : perd son contenu à la coupure d'alimentation
 - Registres, cache, RAM (DRAM)
- **Mémoire non volatile (NVM)** : conserve les données sans alimentation
 - SSD (Flash NAND), disque dur (HDD), bande magnétique
- Le programme d'amorçage (bootstrap) est stocké en ROM/EEPROM (non volatile)
 - Pourquoi ? Le CPU ne peut exécuter que des instructions présentes en mémoire
 - Au démarrage, le chargeur d'amorçage copie le noyau du disque vers la RAM

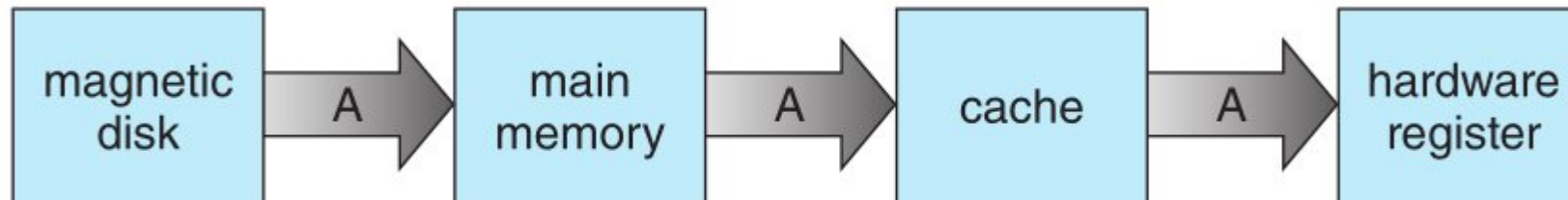
Le caching : principe fondamental

- **Cache** : copie temporaire de données fréquemment utilisées dans un support plus rapide
- **Principe** : si une donnée est utilisée, elle le sera probablement encore bientôt (localité temporelle)
- Si elle est utilisée, les données voisines le seront aussi (localité spatiale)
- La gestion du cache est un problème crucial : taille limitée → politique de remplacement
 - Cohérence de cache (cache coherency) dans les systèmes multiprocesseurs

Un exemple classique : quand vous **ouvrez** un fichier, le SE le **garde** dans un cache mémoire au cas où vous le **réouvririez**.

Migration des données dans la hiérarchie

- **Exemple** : incrémenter un entier A stocké dans un fichier sur disque
- **Étapes** :
 1. Copie du bloc disque vers la RAM (E/S gérée par le SE)
 2. Copie de A de la RAM vers le cache (automatique, matérielle)
 3. Copie de A du cache vers un registre (automatique, matérielle)
 4. Incrémentation dans le registre
- **À la fin** : A existe en 4 copies avec des valeurs possiblement différentes !



Migration of integer A from disk to register

Section 1.4 — Architecture des systèmes informatique

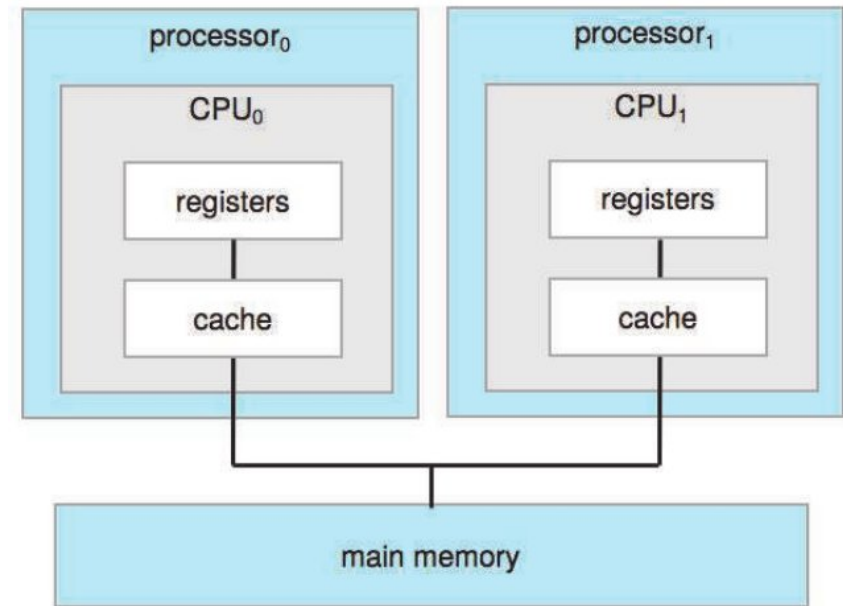
Systèmes monoprocesseur et multiprocesseur

Architecture	CPU généraliste	Spécialisation
Monoprocesseur	1 cœur principal	Processeurs spécialisés (disque, clavier, GPU)
Multiprocesseur SMP	2+ CPU partagent mémoire et bus	Tous les CPU sont pairs
Multicœur	Plusieurs cœurs sur une seule puce	Communication on-chip plus rapide

- **SMP** : Symmetric Multi-Processing — chaque CPU exécute le SE et les processus
- **Avantage principal** : augmentation du débit (mais pas linéaire : $N \text{ CPU} \neq \times N$)

Architecture SMP (Symmetric Multi-Processing)

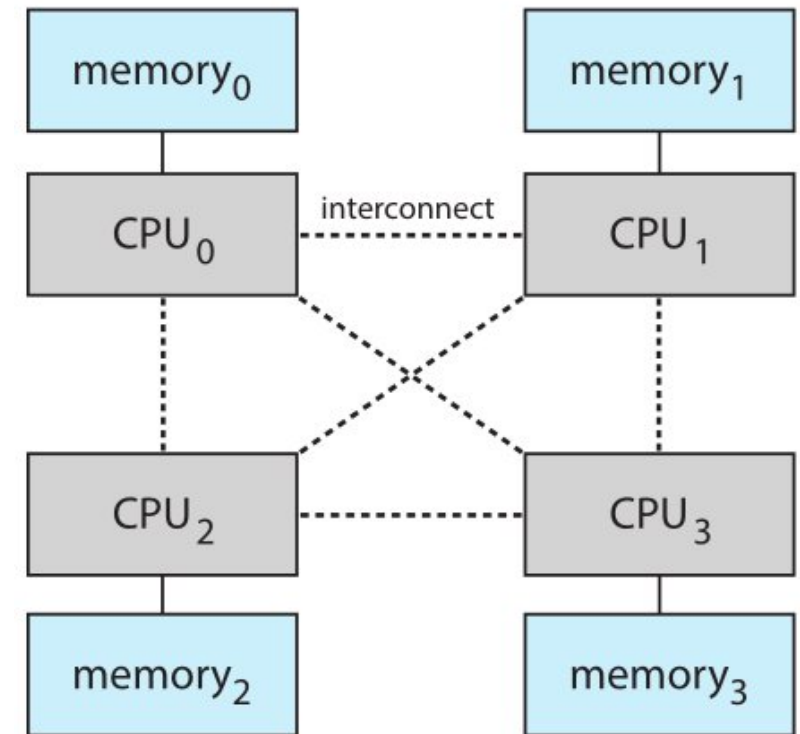
- **CPU 0** : registres, cache privé
- **CPU 1** : registres, cache privé
- **Mémoire physique** : partagée, accessible via le bus système
- **Avantage**: Chaque CPU peut exécuter n'importe quel processus
- **Problème** : si plusieurs CPU accèdent à la même donnée → cohérence de cache



Symmetric multiprocessing architecture

NUMA et systèmes cluster

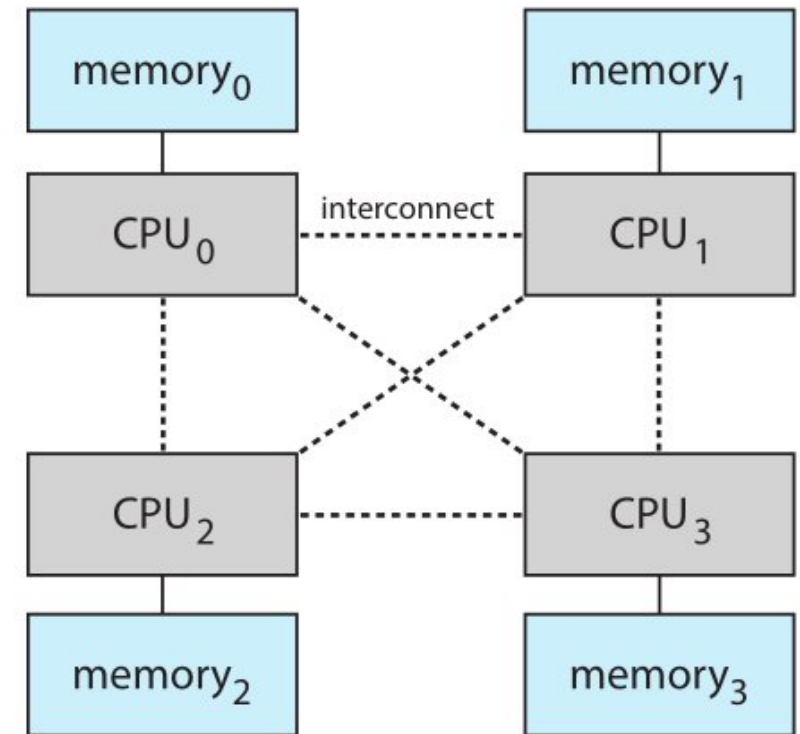
- **NUMA** (Non-Uniform Memory Access) : chaque CPU a sa mémoire locale
 - **Avantage** : accès local rapide, meilleur passage à l'échelle
 - **Inconvénient** : accès à distance plus lent
- **Systèmes cluster** : regroupement de plusieurs nœuds indépendants via un réseau
 - **Haute disponibilité (HA)** : si un nœud tombe, un autre prend le relais
 - **Calcul haute performance (HPC)** : parallélisation d'applications
 - Asymétrique (un serveur en veille) vs. symétrique (tous actifs)



NUMA multiprocessing architecture

NUMA et systèmes cluster

- **NUMA** (Non-Uniform Memory Access) : chaque CPU a sa mémoire locale
 - **Avantage** : accès local rapide, meilleur passage à l'échelle
 - **Inconvénient** : accès à distance plus lent
- **Systèmes cluster** : regroupement de plusieurs nœuds indépendants via un réseau
 - **Haute disponibilité (HA)** : si un nœud tombe, un autre prend le relais
 - **Calcul haute performance (HPC)** : parallélisation d'applications
 - Asymétrique (un serveur en veille) vs. symétrique (tous actifs)



NUMA multiprocessing architecture

Section 1.5 — Fonctionnement du système d'exploitation

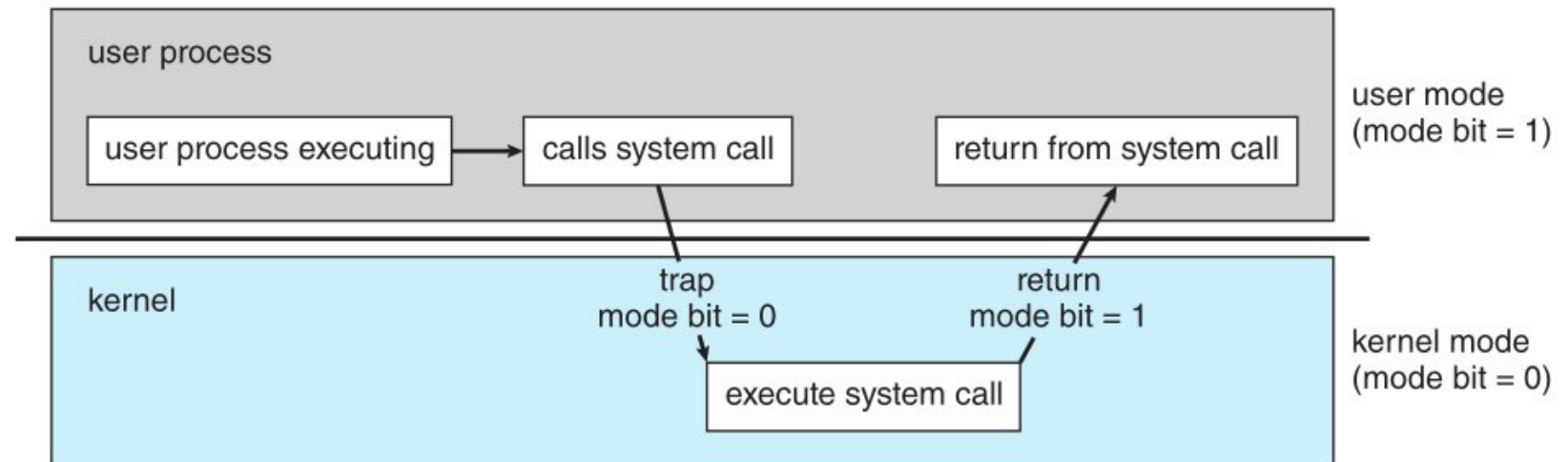
Mode dual : mode utilisateur et mode noyau

- **Mode noyau (mode privilégié, mode 0)** : accès complet au matériel, exécution de toutes les instructions
- **Mode utilisateur (mode 1)** : instructions restreintes, pas d'accès direct au matériel
- **Bit de mode** : indicateur matériel (0 = noyau, 1 = utilisateur)
- Les instructions privilégiées ne peuvent être exécutées qu'en mode noyau
 - **Exemples** : changer le bit de mode, gérer les interruptions, accéder aux périphériques
- En mode utilisateur, toute tentative d'exécuter une instruction **privilégiée** → **trap** → **SE**

Transition mode utilisateur → mode noyau

1. Processus utilisateur s'exécute (mode = 1)
2. Il appelle un appel système (trap / instruction syscall)
3. Le SE exécute le service demandé
4. Le SE restaure le bit de mode à 1 et retourne en mode utilisateur
5. Le processus utilisateur reprend

Important : le basculement de mode est une opération matérielle, pas logicielle — ce qui empêche un programme utilisateur de tricher.



Transition from user to kernel mode.

Protection : mécanismes fondamentaux

- **Protection** : mécanisme contrôlant l'accès des processus/utilisateurs aux ressources
- Moyens de protection :
 - **Mode dual** : empêche les programmes utilisateur d'exécuter des instructions privilégiées
 - **MMU** : isole les espaces mémoire des processus
 - **Timer** : empêche la monopolisation du CPU
 - **Registres de contrôle** : protégés, inaccessibles en mode utilisateur
- **Principe de moindre privilège** : un programme ne doit avoir que les droits nécessaires
- **Exemple** : un programme ne peut pas lire la mémoire d'un autre processus

Les appels système : introduction

- **Appel système (system call)** : requête programmatique d'un service du SE par un processus utilisateur
- **Exemples** : `open()`, `read()`, `write()`, `fork()`, `exit()`
- **Fonctionnement** :
 - Le programme C appelle une fonction de la bibliothèque standard (libc)
 - libc place le numéro de l'appel dans un registre et déclenche un syscall
 - Le noyau exécute le service et retourne le résultat
- **Les appels système sont l'interface entre les applications et le noyau**
- **Détail** : vu en profondeur au Chapitre 3

Le timer (minuterie)

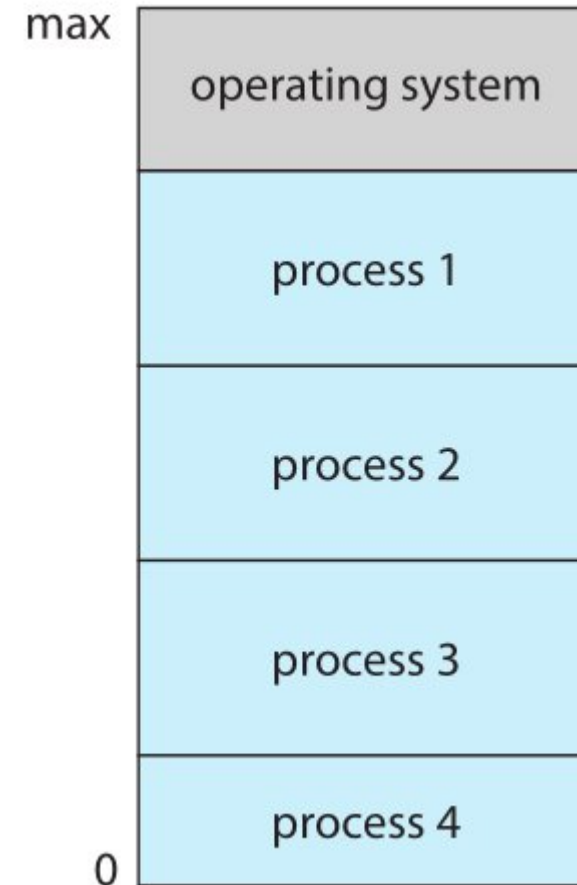
- **Timer** : mécanisme matériel qui interrompt le CPU après une période définie
- **Rôle principal** : empêcher un programme utilisateur de monopoliser le CPU
- **Fonctionnement** :
 - Le SE programme le timer avant de passer en mode utilisateur
 - Le timer compte les ticks d'horloge
 - Quand le compteur atteint zéro → interruption → retour en mode noyau
- **Sur Linux** : paramètre HZ configurable (250 → interruption toutes les 4 ms)
- Les instructions de modification du timer sont privilégiées

Multiprogrammation et multitâche

- **Multiprogrammation** : plusieurs programmes en mémoire simultanément
 - **Objectif** : maximiser l'utilisation du CPU
 - Si un processus attend des E/S, le CPU passe à un autre processus
- **Multitâche** (extension logique) : basculements très fréquents entre processus
 - **Objectif** : temps de réponse rapide pour l'utilisateur
 - Nécessite le timer et l'ordonnancement
- **Résultat** : le CPU n'est jamais inactif (tant qu'il y a un processus prêt)

Organisation mémoire en multiprogrammation

- **0x00000000** (adresse basse)
- SE (noyau) : résident en mémoire
- Processus 1 : espace mémoire alloué
- Processus 2 : espace mémoire alloué
- Processus 3 : espace mémoire alloué
- Processus 4 : espace mémoire alloué
- **0xFFFFFFFF** (adresse haute)



Memory layout for a multiprogramming system

Section 1.6 —Gestion des ressources par le SE

Gestion des processus

- **Processus** : programme en exécution (entité active)
- **Programme** = fichier sur disque (entité passive)
- **Chaque processus possède** : PID, compteur ordinal, registres, mémoire, fichiers ouverts
- **Le SE gère** :
 - Création et suppression de processus
 - Ordonnancement (quel processus obtient le CPU)
 - Synchronisation et communication entre processus
- **Exemple** : deux fenêtres Firefox = deux processus distincts

Gestion de la mémoire et des fichiers

- **Gestion mémoire :**
 - Garder trace des zones mémoire utilisées et par qui
 - Allouer/libérer de l'espace mémoire dynamiquement
 - Gérer les échanges entre RAM et disque (swap)
- **Gestion du système de fichiers :**
 - Abstraction du fichier : unité logique de stockage
 - Organisation en répertoires (arborescence)
 - Contrôle d'accès : qui peut lire/écrire quel fichier
- **Stockage secondaire :** disques, SSD — donnée persistante

Gestion du cache et des entrées-sorties

- **Gestion du cache :**
 - Copier les données fréquemment utilisées dans un support plus rapide
 - Politiques de remplacement : quelle donnée évincer quand le cache est plein ?
 - La cohérence de cache : maintenir la concordance entre niveaux
- **Gestion des E/S :**
 - Composants : tampon (buffer), cache, spooling (file d'attente d'impression)
 - Interface pilote de périphérique (device driver) uniforme
 - Le SE masque les particularités de chaque périphérique

Section 1.7 —Évolution historique des SE

Gestion du cache et des entrées-sorties

Période	Technologie	Mode de fonctionnement
1950s	Tubes à vide, cartes perforées	Traitement par lots (batch) — pas de SE, programme chargé manuellement
1960s	Transistors, circuits intégrés	Multiprogrammation — plusieurs jobs en mémoire, IBM OS/360
1970s	Microprocesseur, temps partagé	Time-sharing — CTSS, MULTICS, Unix (1970) — interactif, multi-utilisateur

- La multiprogrammation est née du besoin de maximiser l'utilisation du CPU
- Le time-sharing a rendu l'informatique interactive

John von Neumann (1903-1957), le père de l'informatique moderne

Chronologie : des années 1980 à aujourd'hui

Période	Innovation	Exemple de SE
1980s	PC, interface graphique (GUI)	MS-DOS, Mac OS
1990s	Internet, open-source	Linux (1991), Windows NT
2000s	Mobile, virtualisation	Android, iOS, VMware
2010s+	Cloud, IoT, containers	AWS, Docker, Kubernetes

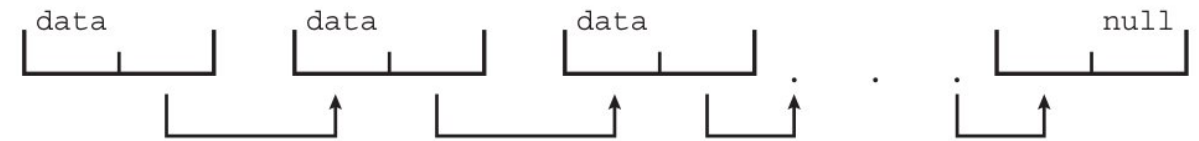
- Loi de Moore : doublement des transistors tous les 18 mois → puissance croissante
- Chaque nouvelle plateforme (mobile, cloud) redéfinit les priorités du SE

La Thinking Machine contenait 65536 processeurs répartis dans un hypercube

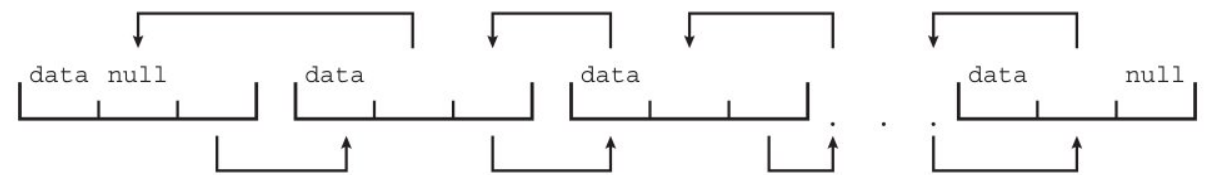
Section 1.8 — Structures de données du noyau

Listes, piles et files d'attente

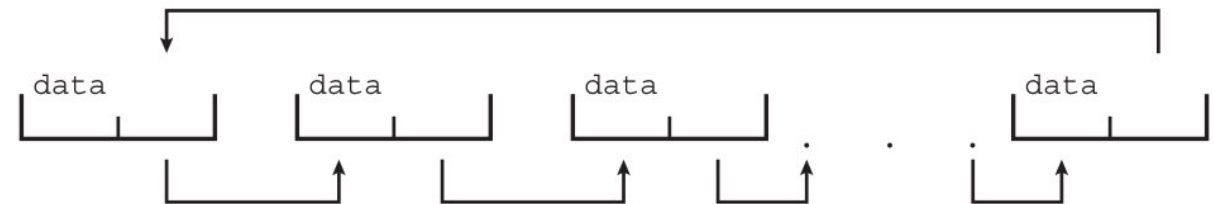
- **Liste chaînée** : collection d'éléments accessibles en séquence
 - **Simple** : chaque élément pointe vers le suivant
 - **Double** : chaque élément pointe vers le précédent et le suivant
 - **Circulaire** : le dernier pointe vers le premier
- **Pile (stack)** : LIFO (Last In, First Out) — push, pop
 - **Utilisation** : appels de fonctions, retour d'interruption
- **File (queue)** : FIFO (First In, First Out) — enfiler, défiler
 - **Utilisation** : files d'attente de processus prêts, spool d'impression



Singly linked list



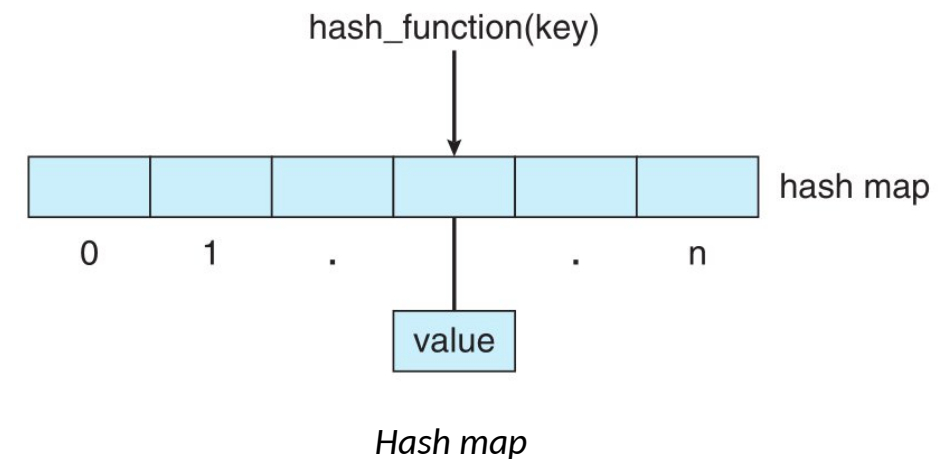
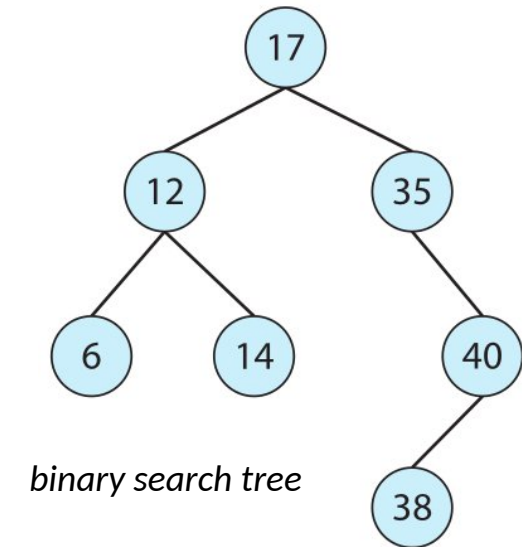
Doubly linked list



Circularly linked list

Arbres, tables de hachage et bitmaps

- **Arbre binaire de recherche (ABR)** : recherche en $O(\log n)$ si équilibré
 - Linux utilise un arbre rouge-noir (**red-black** tree) pour l'ordonnancement
- **Table de hachage (hash map)** : association clé→valeur, accès $O(1)$ en moyenne
 - **Utilisation** : tables de pages mémoire, cache de inodes
- **Bitmap** : chaîne de bits pour représenter l'état d'éléments (libre/occupé)
 - **Utilisation** : gestion des blocs disque, gestion des cadres de page
- **Ces structures sont partout dans le noyau**



Section 1.9 — Environnements de calcul

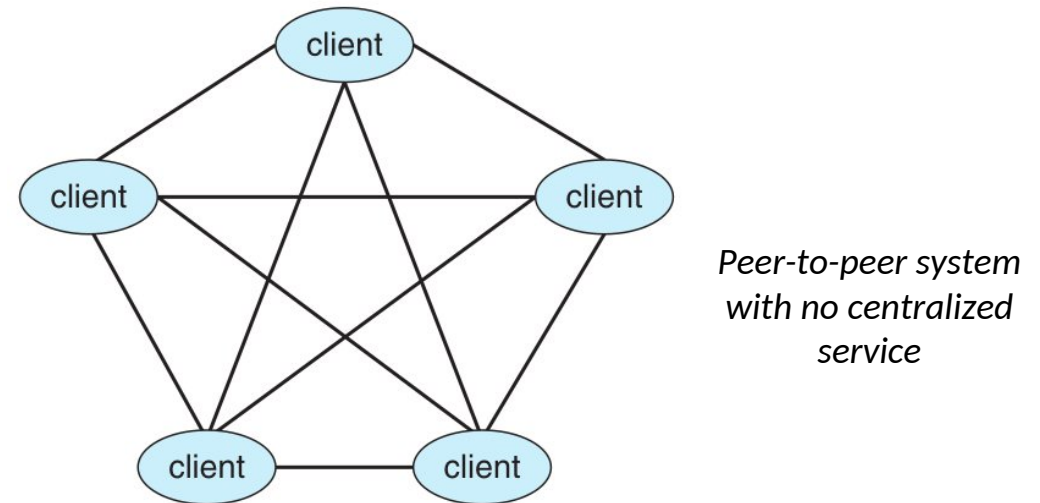
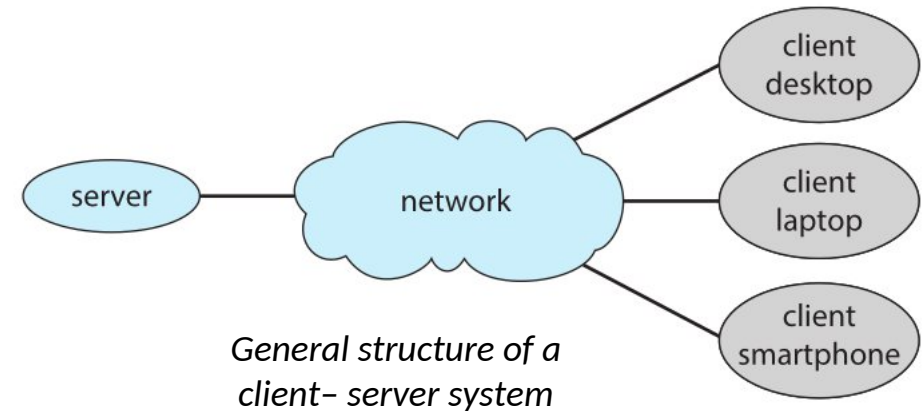
Environnements traditionnel et mobile

Environnement	Contrainte principale	Exemple de SE
Bureau/laptop	Facilité d'utilisation, performance	Windows, macOS, Linux
Serveur	Fiabilité, débit, sécurité	Linux, Windows Server
Mobile	Autonomie, écon. énergie, tactile	Android, iOS
Embarqué	Temps réel, fiabilité, contrainte de taille	VxWorks, FreeRTOS, Linux embarqué

**Le même noyau Linux s'adapte à tous ces environnements
(desktop, serveur, mobile, embarqué)**

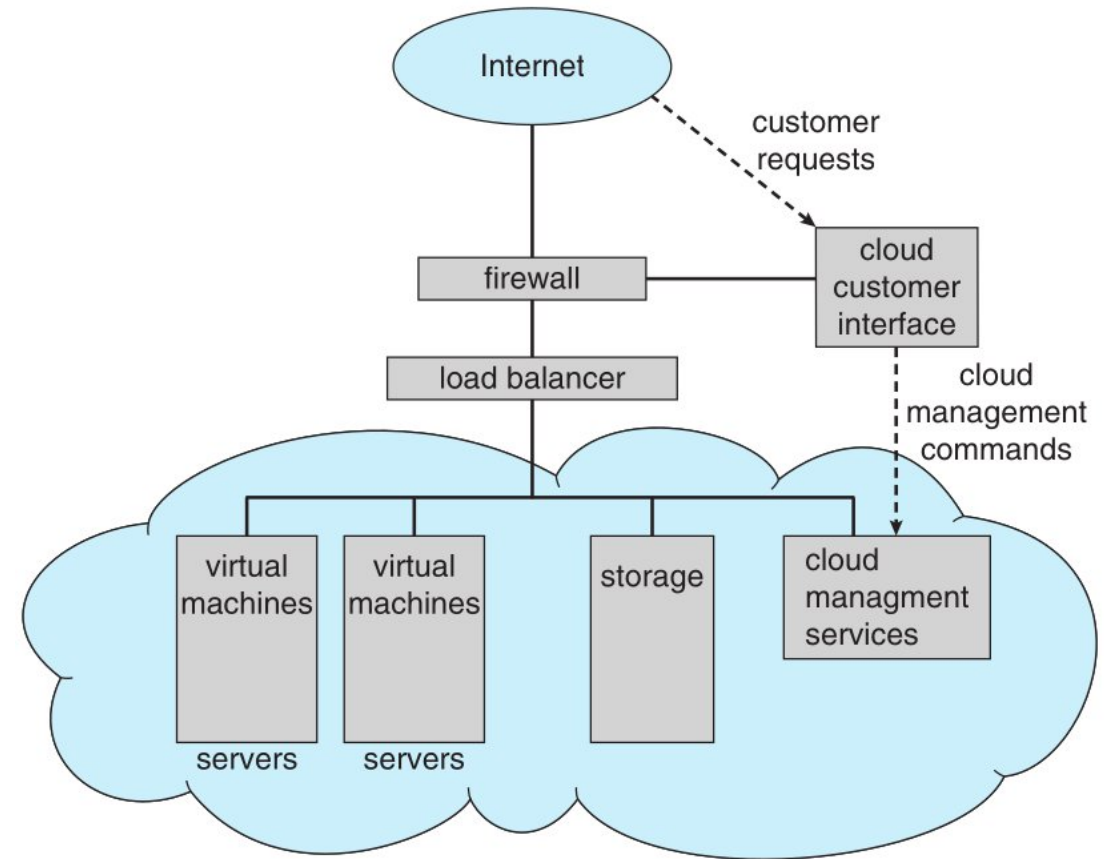
Modèles client-serveur et pair-à-pair

- **Modèle client-serveur** : un serveur central fournit des services, les clients demandent
 - **Exemple** : serveur web (Apache/Nginx) et navigateur client
- **Modèle pair-à-pair (P2P)** : chaque nœud est à la fois client et serveur
 - **Exemple** : protocoles de partage de fichiers (BitTorrent)
- Pas de point central → passage à l'échelle, pas de goulot d'étranglement
- **Découverte de services** : centralisée (annuaire) ou distribuée (broadcast)



Cloud computing

- **Cloud computing** : fourniture de ressources informatiques comme service via le réseau
- **Modèles de service** :
 - **IaaS** (Infrastructure **as a Service**) : serveurs, stockage, réseau — AWS EC2
 - **PaaS** (Platform **as a Service**) : plateforme d'exécution — Google App Engine
 - **SaaS** (Software **as a Service**) : applications — Google Docs, Office 365
- **Types de cloud** : public, privé, hybride
- Le cloud s'appuie sur la virtualisation (plusieurs SE sur un même matériel)



Cloud Computing

Systèmes embarqués et temps réel

- **Système embarqué** : ordinateur dédié à une tâche spécifique
 - **Exemples** : ECU automobile, micro-ondes, drone, montre connectée
 - Ressources limitées, peu ou pas d'interface utilisateur
- **Système temps réel** : contrainte de temps stricte
 - **Temps réel dur** : résultat correct + délai garanti (ABS, airbag)
 - **Temps réel souple** : délai souhaitable mais non fatal (vidéo streaming)

Ces systèmes sont les plus répandus

Des milliards de systèmes embarqués existent

Section 1.10 — Systèmes d'exploitation libres et open-source

Logiciel libre vs. open-source

- **Logiciel libre (free software)** : liberté, pas gratuité
 - **4 libertés** : exécuter, étudier, modifier, redistribuer
 - Défendu par la Free Software Foundation (FSF) — Richard Stallman, 1984
 - **Copyleft** : toute redistribution doit préserver ces libertés
- **Open-source** : code source disponible (pas nécessairement libre)
 - Approche plus pragmatique, centrée sur les bénéfices techniques
- Licences :
 - GPL (GNU General Public License) : copyleft — toute modification doit être GPL
 - BSD / Apache : permissives — intégration possible dans du code propriétaire

GNU/Linux et BSD

- **GNU/Linux (1991)**
 - Noyau Linux (Linus Torvalds) + outils GNU
 - Distributions : Ubuntu, Debian, Fedora, Red Hat, Arch
 - Licence : GPLv2
 - Le SE open-source le plus répandu (serveurs, mobiles, embarqué)
- **BSD (Berkeley Software Distribution, 1978)**
 - Distributions : FreeBSD, OpenBSD, NetBSD
 - Licence BSD (permissive)
 - macOS utilise le noyau BSD (Darwin) comme base
- **Solaris** : UNIX commercial d'Oracle, open-sourcé en 2005 (OpenSolaris → Illumos)

Résumé du chapitre 1

- Le SE est un intermédiaire entre le matériel et les applications (allocateur + contrôleur)
- Les interruptions et le DMA permettent une gestion efficace des E/S
- Le mode dual (utilisateur/noyau) est le fondement de la protection
- Multiprogrammation et multitâche maximisent l'utilisation du CPU
- La hiérarchie mémoire (registre → cache → RAM → disque) exploite la localité
- Architectures : SMP, multicœur, NUMA, clusters
- Le SE gère : processus, mémoire, fichiers, stockage, cache, E/S
- L'évolution des SE reflète l'évolution du matériel
- Les SE libres/open-source (Linux, BSD) permettent l'apprentissage par le code

Préparation au TP1

- Le TP1 vous fait découvrir l'environnement Linux
- Commandes à explorer : `whoami`, `id`, `uname`, `pwd`, `ls`, `cd`, `man`
- Questions de réflexion :
 - Que se passe-t-il quand vous tapez `whoami` ?
 - Quelle est la différence entre *un chemin absolu* et *un chemin relatif* ?
 - Comment obtenir de l'aide sur une commande ?
- **À faire avant le TP:** installer WSL (Windows) ou ouvrir un terminal (Linux/macOS)